SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: **Operating Systems** **COURSE CODE:** **CSA04**

COURSE OUTCOME COVERED

CO5: *Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Total Marks:25**

Annexure A

CONSTRAINT BASED PROBLEM SOLVING

Code of Conduct:

I SIVA SUBRAMANIYAM B/192572089 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
CONSTRAINT BASED PROBLEM SOLVING

S.No.	Question	Mark
1	A server with a total storage capacity of 50 GB needs to effectively manage space for rapidly growing system logs, user files, and application data. Analyze the scenario and design an optimized Linux file system layout, incorporating suitable partitioning techniques and storage management tools (such as log rotation), to ensure efficient utilization of disk space and maintain system stability. The solution must ensure that log files do not exceed 10 GB and that critical services continue to operate without interruption due to disk space exhaustion.	
2	A Linux system handling high CPU workloads suffers from frequent context switching delays. Given that hardware upgrades are not possible and real-time processing must be supported, evaluate kernel-level optimization techniques such as scheduler tuning and the use of kernel modules—to improve overall system performance.	
3	An administrator must process 1 million log entries per day using only shell scripting. Analyze the problem and design an efficient solution using optimized Linux commands that ensures the script completes within 2 minutes. Justify the choice of commands used for achieving high performance	
4	A Linux system runs multiple background services, where a faulty process is consuming 90% of CPU resources. Without restarting the system and while ensuring critical services remain unaffected, analyze the situation and propose a process management strategy using appropriate Linux tools to identify, monitor, and control the faulty process.	
5	A physical server must host 5 virtual machines with a total available RAM of 16 GB. Each VM requires a minimum of 2 GB, while the host operating system needs 4 GB. Analyze the scenario and propose an efficient resource allocation strategy using virtualization concepts to ensure all virtual machines operate smoothly under the given constraints.	
6	A Linux server has 16 GB RAM and runs several applications simultaneously. During peak hours, the system begins using swap heavily even though some applications appear idle. Analyze the memory-management behavior and recommend Linux commands and configuration techniques to identify the cause and optimize memory utilization.	
7	A Linux system contains thousands of files distributed across multiple directories. A backup process is taking several hours to complete. Analyze the directory structure and design an efficient command-line strategy using suitable Linux utilities to identify large files, duplicate files, and unnecessary temporary data.	
8	A Linux server has four CPU cores and supports multiple users running computationally intensive programs. Analyze how Linux scheduling manages these processes and propose a strategy using process priorities and CPU-affinity techniques to improve responsiveness for critical applications.	
9	A system administrator discovers that the `/home` directory is consuming almost all available disk space while `/var` has significant unused space. Analyze the problem and design a suitable storage-management strategy that prevents user files from affecting system services.	

10	A Linux system becomes progressively slower after running continuously for several weeks. No hardware failure is reported. Analyze possible causes involving memory, processes, disk usage, and system resources, and propose a systematic troubleshooting procedure using appropriate Linux commands.	

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Optimized Linux File System Layout for a 50 GB Server

Introduction

A server has a total storage capacity of **50 GB** and must store three major types of data:

1. System logs
2. User files
3. Application data

The main challenge is that system logs are growing rapidly. If logs consume the entire disk, critical Linux services may stop because the operating system needs free space for normal operation. Therefore, the storage must be divided logically and appropriate log-management techniques must be used.

The assessment specifically requires that **log files must not exceed 10 GB** and that critical services should continue operating without interruption due to disk exhaustion.

Proposed Partitioning

A suitable 50 GB layout is:

Partition	**Size**	**Purpose**
/	10 GB	Operating system and system programs
/var	10 GB	Logs, cache and variable application data
/home	15 GB	User files
/opt	7 GB	Application software
/tmp	2 GB	Temporary files
Swap	6 GB	Virtual memory
Total	**50 GB**	

The important design decision is to keep /var separate from /home.

The /var/log directory contains many Linux system and application logs. If /home and /var were part of the same unrestricted filesystem, users could consume the entire disk and leave insufficient space for system services.

Log Rotation

Linux provides the logrotate utility for automatically managing logs. A configuration can be created using:

sudo nano /etc/logrotate.d/mylogs

Example:

```
/var/log/*.log    {
 daily
 rotate  7
compress
missingok
```

```
notifempty
size 100M
}
```

Here:

- daily rotates logs every day.
- rotate 7 retains seven rotated copies.
- compress compresses old logs.
- missingok prevents errors if a log does not exist.
- notifempty prevents empty logs from being rotated.
- size 100M allows rotation when the log becomes large.

Monitoring Disk Usage

The administrator can monitor filesystem usage using:

```
df -h
```

To check the size of the log directory:

```
du -sh /var/log
```

To identify large log files:

```
sudo du -ah /var/log | sort -rh | head -20
```

Preventing Disk Exhaustion

The following strategy can be adopted:

1. Keep /var separate.
2. Restrict the size of logs.
3. Rotate logs regularly.
4. Compress old logs.
5. Delete obsolete logs after the retention period.
6. Monitor filesystem usage.
7. Configure alerts when storage reaches a critical threshold.
8. Keep sufficient free space for essential services.

Advantages

- Prevents logs from consuming the complete disk.
- Protects critical system services.
- Improves storage utilization.
- Makes old logs easier to manage.
- Reduces storage requirements through compression.
- Makes troubleshooting easier because historical logs are retained.

Conclusion

The best solution is to use separate partitions for the operating system, logs, user data and applications. /var should be isolated and managed using logrotate. With a **10 GB limit for logs**, regular rotation, compression and monitoring, the server can maintain system stability without disk-space exhaustion.

---

2. Reducing Context Switching Delays in Linux

Introduction

The second problem describes a Linux system handling high CPU workloads where frequent context switching causes performance delays. Hardware upgrades are not possible, and the system must support real-time processing. Therefore, kernel-level techniques such as **scheduler tuning, process priorities, CPU affinity and kernel-module management** must be considered.

What is Context Switching?

A context switch occurs when the CPU changes from executing one process or thread to another. During a context switch, Linux must save the state of the currently running process and restore the state of another process.

Excessive context switching causes:

- CPU overhead
- Cache disruption
- Reduced application performance
- Increased response time
- Delays in real-time applications

Causes of Frequent Context Switching

Possible causes include:

1. Too many active processes.
2. Large numbers of threads.
3. Poor process priorities.
4. Excessive interrupts.
5. CPU-intensive background processes.
6. Poor CPU scheduling configuration.

Scheduler Tuning

Linux uses scheduling policies to decide which process should receive CPU time. Important scheduling policies include:

- SCHED_OTHER – normal scheduling.
- SCHED_FIFO – first-in-first-out real-time scheduling.
- SCHED_RR – real-time round-robin scheduling.

For real-time workloads, critical processes can be assigned suitable real-time scheduling policies.

Process Priority

- The nice value can influence process scheduling.
- A process can be started with:
- nice -n 10 ./program
- An existing process can be modified using:
- sudo renice 10 -p PID
- Lower-priority background processes can be given larger nice values so that critical applications receive more CPU attention.

CPU Affinity

CPU affinity allows a process to run only on selected CPU cores.

Example:

taskset -c 0,1 ./critical_application

For an existing process:

taskset -cp 0,1 PID

This can reduce unnecessary movement of processes between CPU cores.

Kernel Module Management

- Kernel modules provide additional functionality to the Linux kernel.
- Loaded modules can be examined using:
- lsmod
- Information about a module can be obtained using:
- modinfo module_name
- Unnecessary modules should be avoided where appropriate because unnecessary kernel functionality can increase resource usage and complexity.

Performance Monitoring

The administrator should monitor the system before and after optimization.

Useful commands include:

top

vmstat 1

pidstat 1

These commands help determine whether CPU utilization and scheduling behavior have improved.

Recommended Strategy

Identify CPU-heavy processes

↓

Monitor context switching

↓

Adjust process priorities

↓

Apply CPU affinity

↓

Use suitable scheduling policy

↓

Review unnecessary kernel modules

↓

Monitor performance again

Conclusion

Without hardware upgrades, Linux kernel-level optimization can reduce unnecessary context switching. Scheduler tuning, process priority management, CPU affinity and careful kernel-module management can improve CPU utilization and provide better response time for real-time applications.

---

3. Processing 1 Million Log Entries Using Shell Scripting

Introduction

The administrator must process **1 million log entries per day** using only shell scripting. The script must complete within **2 minutes**. Therefore, efficient Linux text-processing commands must be used instead of slow line-by-line shell processing.

Problem Analysis

Processing one million records is a large workload. A poorly designed script could take several minutes because every line may result in multiple external command executions.

For example, repeatedly executing commands inside a loop can create significant overhead. Therefore, commands capable of processing large amounts of text efficiently should be preferred.

Suitable Linux Commands

Important commands include:

- grep
- awk
- cut
- sort
- uniq
- wc

Counting Log Entries

wc -l application.log

This counts the number of lines in the file.

Counting Errors

```
grep -c "ERROR" application.log
```

Using AWK

- awk is especially useful because it can search, filter and process records in a single operation.
- awk '/ERROR/ {count++} END {print count}' application.log

Counting Warnings

```
awk '/WARNING/ {count++} END {print count}' application.log
```

Extracting Fields

- Suppose the log format contains fields separated by spaces:
- awk '{print $5}' application.log

Finding Unique Values

```
awk '{print $5}' application.log | sort | uniq -c
```

Example Optimized Script

```
#!/bin/bash
LOGFILE="application.log"
echo "Total log entries:"
wc -l < "$LOGFILE"
echo "ERROR entries:"
awk '/ERROR/ {count++} END {print count}' "$LOGFILE"
echo "WARNING entries:"
awk '/WARNING/ {count++} END {print count}' "$LOGFILE"
```

Measuring Execution Time

- time ./process_logs.sh
- The administrator can use the result to determine whether the script satisfies the two-minute requirement.

Performance Improvements

The following techniques improve performance:

1. Use awk instead of unnecessary shell loops.
2. Avoid executing external commands for every record.
3. Process the file sequentially.
4. Avoid unnecessary output.
5. Use pipelines carefully.

6. Extract only required fields.
7. Avoid repeatedly opening the same file.

Conclusion

For one million log records, efficient utilities such as awk, grep, sort, uniq and wc are preferable to inefficient shell loops. Proper command selection and performance measurement can help the administrator meet the **two-minute execution constraint**.

---

4. Managing a Faulty Process Consuming 90% CPU

Introduction

A Linux system has several background services, but one faulty process is consuming approximately **90% of CPU resources**. The administrator must control the process without restarting the system and must ensure that critical services are not affected.

Step 1 – Identify the Faulty Process

Use:

top

The administrator can sort processes by CPU usage.

Another method is:

ps aux --sort=-%cpu | head

This displays the processes using the most CPU.

Step 2 – Find the Process ID

Suppose the faulty process has PID 2450.

The process can be monitored using:

top -p 2450

or:

pidstat -p 2450 1

Step 3 – Reduce CPU Priority

- Instead of immediately terminating the process, its priority can be reduced:
- sudo renice 10 -p 2450
- This allows more CPU time to be available for important services.

Step 4 – Restrict CPU Affinity

- The process can be restricted to a particular CPU:
- sudo taskset -cp 0 2450
- This prevents the process from consuming CPU resources across all available cores.

Step 5 – Terminate Gracefully

- If the process remains faulty:
- kill 2450

- The normal kill command sends a termination signal and gives the process an opportunity to exit cleanly.
- If it refuses to terminate:
- kill -9 2450
- This should be considered a last resort.

Step 6 – Verify Critical Services

- After controlling the process:
- systemctl status service_name
- The administrator should verify that critical services continue running.

Overall Strategy

Identify process

↓

Check PID

↓

Monitor CPU usage

↓

Reduce priority

↓

Restrict CPU affinity

↓

Terminate if required

↓

Verify critical services

Conclusion

The safest approach is gradual intervention. The administrator should first identify and monitor the process, then reduce its priority or CPU affinity. Only if these techniques fail should the process be terminated. This meets the requirement of maintaining critical services without rebooting the server.

---

5. RAM Allocation for 5 Virtual Machines

Given Information

The physical server has:

- **16 GB total RAM**
- **5 virtual machines**
- Each VM requires at least **2 GB**
- Host operating system requires **4 GB**

Calculation

Memory required by all VMs:

$5 \times 2\ GB = 10\ GB$

Memory required by host:

4 GB

Total minimum:

10 GB + 4 GB = 14 GB

Available:

16 GB

Remaining:

16 GB - 14 GB = 2 GB

Therefore, all five VMs can operate at their minimum requirement.

Proposed Allocation

Resource	Allocation
Host OS	4 GB
VM1	2 GB
VM2	2 GB
VM3	2 GB
VM4	2 GB
VM5	2 GB
Reserve	2 GB
Total	**16 GB**

Why Keep 2 GB as Reserve?

The virtualization host itself requires memory for:

- Hypervisor operations
- Network operations
- Storage operations
- Background services
- Virtualization overhead

Therefore, allocating the entire remaining memory immediately to VMs could cause memory pressure.

Virtualization Techniques

The administrator can use:

1. Memory Allocation

Initially allocate 2 GB to each VM.

2. Memory Ballooning

Where supported, memory ballooning allows unused VM memory to be reclaimed by the host.

3. Monitoring

VM memory usage should be continuously monitored.

4. Avoid Excessive Overcommitment

The administrator should not allocate more memory than the physical system can reliably support.

Conclusion

The configuration is feasible because the five VMs require only **10 GB**, while the host requires **4 GB**, resulting in **14 GB** total. The remaining **2 GB** should be retained as a safety margin to ensure smooth operation.

---

6. Heavy Swap Usage on a 16 GB Linux Server

Introduction

The Linux server has **16 GB RAM**, but during peak hours it starts using swap heavily even though some applications appear idle. The problem requires analysis of Linux memory-management behavior and the use of commands and configuration techniques to optimize memory utilization.

Linux Memory Management

Linux divides memory among:

- Running applications
- Kernel
- Buffers
- File cache
- Shared memory
- Inactive pages

Linux may move less-active memory pages to swap so that RAM can be used more efficiently. Therefore, simply seeing some swap usage does not always mean that the system has a serious problem. However, **continuous swap-in and swap-out** can indicate memory pressure.

Step 1 – Check Memory

free -h

This provides information about RAM and swap.

Step 2 – Check Swap

swapon --show

Step 3 – Monitor Memory Activity

vmstat 1

The si and so values are especially useful.

- si = swap in
- so = swap out

High continuous values indicate memory pressure.

Step 4 – Find Memory-Heavy Processes

```
ps aux --sort=-%mem | head
```

or:

```
top
```

This helps identify applications consuming large amounts of memory.

Step 5 – Check Swappiness

```
cat /proc/sys/vm/swappiness
```

The administrator can temporarily change it:

```
sudo sysctl vm.swappiness=10
```

For persistent configuration:

```
sudo nano /etc/sysctl.conf
```

Add:

```
vm.swappiness=10
```

Then apply:

```
sudo sysctl -p
```

Additional Optimization

The administrator should:

1. Identify memory-hungry applications.
2. Stop unnecessary processes.
3. Check for memory leaks.
4. Increase application efficiency.
5. Avoid unnecessary background services.
6. Monitor swap activity during peak hours.

Conclusion

Heavy swap usage should be analyzed rather than immediately assuming that all swap usage is harmful. free, vmstat, top, ps, and swapon help identify the cause. Appropriate memory-management configuration and application optimization can reduce excessive swapping.

---

7. Optimizing a Slow Backup Process

Introduction

The Linux system contains thousands of files distributed across several directories, and the backup process takes several hours. Before modifying the backup strategy, the administrator should identify large files, duplicate files and unnecessary temporary data.

Step 1 – Analyze Directory Sizes

du -sh /*

For /home:

du -sh /home/*

This helps locate directories consuming significant storage.

Step 2 – Find Large Files

- For files larger than 500 MB:
- sudo find / -type f -size +500M -exec ls -lh {} \;
- The administrator can inspect these files and determine whether they are required.

Step 3 – Find Temporary Files

- find /tmp -type f -mtime +7
- This identifies temporary files older than seven days.
- They should only be deleted after confirming that they are no longer required.

Step 4 – Identify Duplicates

- Checksums can be used to identify files with identical contents:
- md5sum file1 file2
- For many files:
- find /home -type f -exec md5sum {} \; | sort
- Files with matching checksums can be investigated as possible duplicates.

Step 5 – Clean Storage

After verification:

- Remove unnecessary temporary files.
- Remove duplicate files.
- Archive old files.
- Exclude unnecessary cache directories.
- Compress suitable data.

Step 6 – Improve Backup

Instead of backing up every file every time, the administrator can use:

- Incremental backups
- Differential backups
- Compression
- Exclusion rules

Conclusion

The backup delay can be reduced by identifying and removing unnecessary data before backup. du, find, sort and checksum utilities provide an efficient command-line strategy for analyzing storage and reducing backup size.

8. CPU Scheduling on a Four-Core Linux System

Introduction

The Linux system contains **four CPU cores** and multiple users are running computationally intensive applications. Linux scheduling determines which processes receive CPU time. Critical applications need better responsiveness, so process priorities and CPU affinity can be used.

Linux Scheduling

The Linux scheduler determines which runnable task should execute on a CPU.

When many applications compete for CPU resources, scheduling becomes important.

Process Priority

- The administrator can examine process priorities:
- ps -eo pid,ni,comm
- A background process can be given lower priority:
- sudo renice 10 -p PID
- A critical process can be given a more favorable priority where appropriate:
- sudo renice -5 -p PID

CPU Affinity

CPU affinity controls which processors a process can use.

For example:

taskset -c 0,1 ./critical_app

This allows the critical application to execute on cores 0 and 1.

For an existing process:

taskset -cp 0,1 PID

Proposed Allocation

A possible strategy is:

CPU 0 → Critical application

CPU 1 → Critical application/support tasks

CPU 2 → User applications

CPU 3 → User applications

This is an illustrative strategy; actual allocation should be based on workload and measured CPU utilization.

Benefits

- Improves responsiveness.
- Reduces unnecessary CPU migration.
- Protects important processes from CPU starvation.
- Provides better workload organization.

- Helps control resource-intensive background applications.

Conclusion

Linux scheduling combined with nice, renice and taskset provides administrators with practical methods to prioritize important applications and control CPU placement. This can improve responsiveness on a four-core system without requiring hardware upgrades.

---

9. home Consuming Almost All Disk Space

Introduction

The Linux server has a storage imbalance: /home is almost full while /var contains significant unused space. If user files consume all available disk space, system services that require /var may fail. Therefore, user data must be separated and controlled.

Step 1 – Check Filesystem Usage

df -h

This identifies which filesystem is full.

Step 2 – Identify Large Users

du -sh /home/*

For individual large files:

find /home -type f -size +1G -ls

Step 3 – Clean Unnecessary Data

The administrator can identify:

- Old downloads
- Temporary files
- Duplicate files
- Old archives
- Unnecessary application data

Unnecessary data should be removed only after verification.

Step 4 – User Quotas

- Disk quotas can prevent individual users from consuming unlimited storage.
- For example:
- sudo quotaon /home
- Appropriate user quotas can then be configured.

Step 5 – Storage Separation

A good filesystem architecture is:

/ → Operating system

/var → Logs and application variable data

/home → User files

Using separate partitions or LVM logical volumes prevents user storage from directly consuming space needed by other filesystems.

Step 6 – LVM

Logical Volume Manager provides flexible storage management. Logical volumes can be resized when necessary, subject to filesystem and storage constraints.

Conclusion

The administrator should control /home using quotas, cleanup policies and storage separation. This ensures that large user files do not consume the space required by system services and improves overall system reliability.

---

10. Systematic Troubleshooting of a Slow Linux System

Introduction

The Linux system becomes progressively slower after running continuously for several weeks. No hardware failure has been reported. Therefore, the administrator should investigate software and resource-related causes involving **CPU, memory, processes, disk usage and system services**.

Step 1 – Check CPU Usage

Start with:

top

or:

htop

Look for processes consuming unusually high CPU.

Another useful command is:

ps aux --sort=-%cpu | head

Step 2 – Check Memory

free -h

Check whether RAM is nearly full and whether swap is heavily used.

Then:

vmstat 1

This helps identify memory pressure and system activity.

Step 3 – Check Disk Space

df -h

A filesystem that is nearly full can cause system problems.

Find large directories:

du -sh /*

Step 4 – Check Disk I/O

Use:

iostat

High disk utilization may indicate that the system is waiting heavily for storage operations.

Step 5 – Analyze Processes

- CPU-heavy processes:
- ps aux --sort=-%cpu | head
- Memory-heavy processes:
- ps aux --sort=-%mem | head
- The administrator should investigate processes that continuously consume large resources.

Step 6 – Check Failed Services

systemctl --failed

This identifies services that have failed.

Step 7 – Check System Errors

journalctl -p err

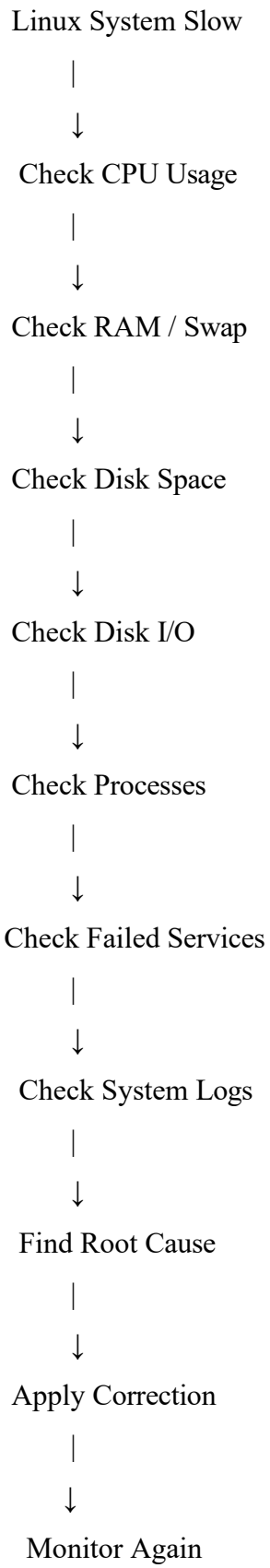
System logs may reveal:

- Service failures
- Resource problems
- Application errors
- Storage problems
- Kernel errors

Step 8 – Check Long-Running Processes

- ps -eo pid,etime,cmd --sort=etime
- Long-running processes should be investigated for abnormal resource usage or possible memory leaks.

Troubleshooting Flow



Corrective Actions

Depending on the root cause:

- Stop unnecessary processes.
- Reduce CPU priority of background processes.
- Clean unnecessary files.
- Rotate large logs.
- Optimize memory usage.
- Restart only affected services when necessary.
- Correct failed services.
- Investigate possible memory leaks.
- Monitor the system after corrective action.

Conclusion

A progressively slower Linux system should not immediately be rebooted. A systematic approach using top, free, vmstat, df, du, ps, iostat, systemctl, and journalctl can identify the actual source of the problem. Corrective action should then be applied specifically to the affected resource. This approach improves system stability and minimizes service interruption.
